

A Lightweight Cryptographic State Consistency and Recovery Mechanism for Secure Network Communication

GHOSH, TUTUL

*Department of Computer Science and Engineering
American International University-Bangladesh
23-50879-1@student.aiub.edu*

MD Tanvir Sajib

*Department of Computer Science and Engineering
American International University-Bangladesh
22-46485-1@student.aiub.edu*

Tarikul Islam

*Department of Computer Science and Engineering
American International University-Bangladesh
20-43662-2@student.aiub.edu*

Abstract—Secure network communication protocols depends on synchronized cryptographic state such as session keys, nonces counter, sequence numbers, and epoch counters between the two endpoint devices. When a common network events such as packet loss, reordering, duplication, delay, connection interruption, or endpoint restart occur, the cryptographic states maintained by two legitimate endpoints may differ which leads to authentication failures, message rejection, or maybe potential security vulnerabilities. While the existing research has addressed desynchronization within specific authentication protocols (RFID) or by protocol-specific mechanisms (IPsec anti-replay windows, DTLS epoch management), a systematic, protocol-independent approach for detection, classification, and recovering from network-induced cryptographic state divergence remains overlooked. This paper presents a lightweight authenticated state-consistency and recovery mechanism (LSCR) designed to operate as a protocol independent layer. We propose a framework of cryptographic state divergence causes, a state-consistency verification protocol using authenticated state digests, a divergence classification scheme which separate recoverable from irrecoverable states, and a authenticated recovery process that minimizes overhead while preventing unsafe progression. This paper implement and evaluate the mechanism across 12 network scenarios using controlled network simulation with Linux tc/netem which measure security metrics (replay acceptance rate, nonce reuse count, unsafe recovery rate), reliability metrics (recovery success rate, session failure rate), and performance metrics (recovery latency, communication overhead, computational cost). Experimental results across 480 experiment runs (12 scenarios $\times$ 10 trials $\times$ 2 configurations $\times$ 2 roles) confirm that LSCR maintains all security invariants (zero nonce reuse, zero replay acceptance, zero unsafe recoveries) while imposing minimal performance overhead (11.5 encryption latency increase, 0.1 memory increase, 1.2 percentage point CPU increase). Under packet loss scenarios (1–10), LSCR achieves comparable or slightly improved message delivery rates relative to the baseline. This work contributes a generalized framework of network induced cryptographic state divergence, a lightweight authenticated state-consistency mechanism, a secure recovery decision process, and a controlled experimental evaluation framework that jointly assesses security, availability, and performance.

Index Terms—cryptographic state synchronization, state desynchronization, replay protection, nonce management, session

recovery, state consistency, secure communication protocols

I. INTRODUCTION

Secure network communication requires two communicating endpoint devices that maintain consistent cryptographic state through a full session. This state includes multiple inter-related variables, such as session keys derived from authenticated encryption, sequence numbers, increasing nonces or initialization vectors, epoch identifiers, and session metadata binding. [] The importance of cryptographic state consistency is well established. For example, if a sender encrypts a message using nonce 'n' under a key 'k', but the receiver expects nonce 'm' which is not the same as the sender's, then the receiver will fail to authenticate and decrypt the message, even if both the sender and receiver are legitimate and the message has not been altered. Similarly, if the receiver's anti-replay window has advanced beyond the sender's current sequence number due to packet loss causing the sender to retransmit with a stale counter.[1], [2]. Many real-world network environments introduce multiple events that can cause such state desynchronization between the endpoints, like packet loss causing the receiver to miss the state advancing message, which creates gaps in the expected sequence. Packet duplication causes the receiver to process the same state multiple times. Packet delay can trigger timeout-based transitions on one side, but the other side will be as it is [3], [4].

Existing security protocols solve these challenges through protocol specific mechanisms. IPsec uses a sliding window for anti-replay protection with configurable window sizes[3]. DTLS 1.3 employs epoch based cipher state management with independent record processing to tolerate packet loss[5]. TLS 1.3 manages session state by key schedule and session ticket mechanism[2]. WireGuard uses a large anti-replay for handshake replay protection[6]. RFID authentication protocols employs various anti desynchronization strategies [7], [8]. However, the mechanism discussed are embedded within their

respective protocols. When cryptographic state desynchronization occurs due to network events, the natural response ranges from silent packet discard or connection termination. Furthermore, recovery from state divergence generally requires re-running the full handshake again or manual intervention with no systematic evaluation of when partial state recovery is safe or a full handshake establishment is needed [9], [10].

To address this problem, this research proposes a framework for cryptographic state divergence management. The proposed approach introduces a framework that classifies cryptographic state divergence according to its causes and severity levels, providing a structured basis for determining appropriate recovery actions. Instead of retransmitting the cryptographic state divergence directly, the framework employs authenticated state digests to verify synchronization status securely. Furthermore, a recovery mechanism is also implemented that attempts authenticated partial synchronization when divergence is limited and trustworthy. When divergence is extensive or potentially malicious, the system initiates a secure fallback procedure involving re-establishment of the cryptographic session through a fresh handshake.

The main novelty of this paper is to develop a generalized framework of cryptographic state divergence caused by network-level events. The framework identifies the key variables that may become inconsistent during communication, such as counters, sequence numbers, and session ID. It also measures the security consequences of these divergences, including authentication failures and synchronization errors. This proposed system also classifies different recovery scenarios and determines the condition under which normal communication can be restored.

A. Research Question

1. Can a lightweight, protocol-independent mechanism detect and recover from state divergence without weakening security guarantees?
2. What is the performance overhead (latency, CPU, memory, communication) of such a mechanism during normal operation?
3. How does the mechanism compare to the baseline fail-closed strategy in terms of security, availability, and performance across varying network conditions?

II. LITERATURE REVIEW

In the transport security domain, TLS 1.3 (RFC 8446) [2] follows a systematic key schedule based on HKDF, which derives distinct keys for different protocol phases. The key schedule inherently enforces state transitions. The TLS 1.3 state machine is deliberately simplified compared to earlier versions, removing legacy features like renegotiation that had historically introduced state management vulnerabilities [2], [12]. Thus, TLS operates over TCP and delegates packet-level reliability to the transport layer, making it unsuitable as a model for environments with unreliable transport.

DTLS 1.3 [5] extends TLS 1.3 to operate over unreliable datagram transport (UDP) protocols. DTLS focuses on the

challenges of packet loss and reordering through epoch-based cipher state management. This mechanism uses encrypted sequence numbers for privacy and independent record processing which does not require cryptographic chaining between records [5]. Each DTLS record is independently decryptable, and the receiver maintains per-epoch sliding windows for anti-replay protection. This design represents the most mature approach to cryptographic state management over unreliable transport in a standardized protocol.

WireGuard [6] takes a simple approach, using a 64-bit counter for nonces with ChaCha20Poly1305 and a sliding window of approximately 2000 entries for anti-replay. Its handshake (based on the Noise IK pattern [13]) uses TAI64N timestamps to prevent handshake replay. WireGuard's state management is notable for its simplicity and its "silent by default" property—unauthenticated packets are silently discarded without state allocation on the responder side.

The Signal Protocol's Double Ratchet [14] provides a Modern approach to continuous key synchronization, where each message derives new key material through both symmetric-key evolution and asymmetric evolution (DH exchanges). This system provides both forward secrecy and post-compromise security. However, the Double Ratchet requires message ordering within each ratchet step and becomes complex to implement correctly in environments with significant message loss or reordering.

IPsec ESP [3] defines a sliding-window anti-replay mechanism where the receiver maintains a window of versions, provides numbers. Packets outside the window (below the left edge) are discarded as potential replays. The minimum window size is 32 packets, with 64 recommended as the default [3]. This mechanism has been adopted as the basis for DTLS anti-replay protection [5]. However, the sliding window approach can cause legitimate packet rejection when network reordering exceeds the window size, particularly in high-speed or multipath environments [3], [4].

In RFID authentication, several desynchronization attacks have been reported in the literature [11]. the dual-state approach where the server stores both the current and previous key versions—provides a simple recovery mechanism at the cost of increased storage and a vulnerability window during which the old key remains valid [7], [8]. More sophisticated approaches use hash chains to enable independent key refresh, reducing the dependence on synchronized state updates [15].

In the Noise Protocol Framework [13], recovery from connection loss requires re-executing the handshake pattern. There is no built-in mechanism for resuming a disrupted session with the same keys. Applications using Noise (such as WireGuard) implement their own session lifecycle management, typically triggering new handshakes when timers expire or when authentication failures accumulate.

Dynamic session key protocols for IoT environments, such as DSEKP [21], use HMAC-based key derivation functions (HKDF) to derive per-session keys, providing forward secrecy with symmetric-key-only operations. These protocols often incorporate nonce-based challenge-response mechanisms for

session freshness verification and hash-chaining for independent key recovery [21], [22].

For IoT and constrained environments, lightweight authentication protocols have introduced edge-assisted synchronization mechanisms, such as L-TESLA-CoAP [23], that enable continuous authentication without the overhead of full DTLS/TLS, using rotating pseudonyms and delayed key disclosure to prevent replay and impersonation attacks.

III. RESEARCH GAP

Table I summarizes the limitations of existing approaches and highlights the research gap addressed in this work.

TABLE I
COMPARISON OF STATE DESYNCHRONIZATION DETECTION AND RECOVERY MECHANISMS

Paper	Detection	Recovery	Key Limitation
IPsec ESP	Sliding window	None (packet discard)	No state recovery, discard-oriented
RFID Authentication	Dual state check	Old/new key	Protocol specific, no general network-induced desynchronization analysis
RFID Anonymous Authentication	Dual state check	Old/new key	Protocol specific, no general network model
Signal Double Ratchet	Implicit	Ratchet forward	Requires ordering within ratchet steps
WireGuard	Sliding window	Rehandshake	No partial recovery, silent packet discard
TLS 1.3	Implicit	Reconnect	TCP-based, no native datagram support
Noise Protocol Framework	Authentication failure	Rehandshake	No partial session recovery, relies on full rehandshake
AES-GCM-SIV	N/A	Deterministic encryption	Primitive-level solution, not a protocol-level recovery mechanism
DTLS 1.3	Sliding window	Retransmission	Protocol specific, no explicit divergence classification
LTESLACoAP	Delayed disclosure	Edge-assisted synchronization	Domain specific, requires trusted edge
Automata-Based State Machine Analysis [18]	Automata learning	None	Detection oriented, does not provide cryptographic state recovery
PUF-Based Authentication [16]	Fuzzy extractor	Hardware-rooted recovery	Hardware dependency, no general network model
Opossum Attack [17]	Attack-based analysis	None	Focuses on exploitation, does not provide secure state recovery

IV. EXISTING FRAMEWORK

For the baseline model the authors consider a client server communication model where two endpoint devices communicate over an unreliable network channel. The baseline architecture provides authenticated encryption using standard cryptographic primitives. We intentionally use a controlled,

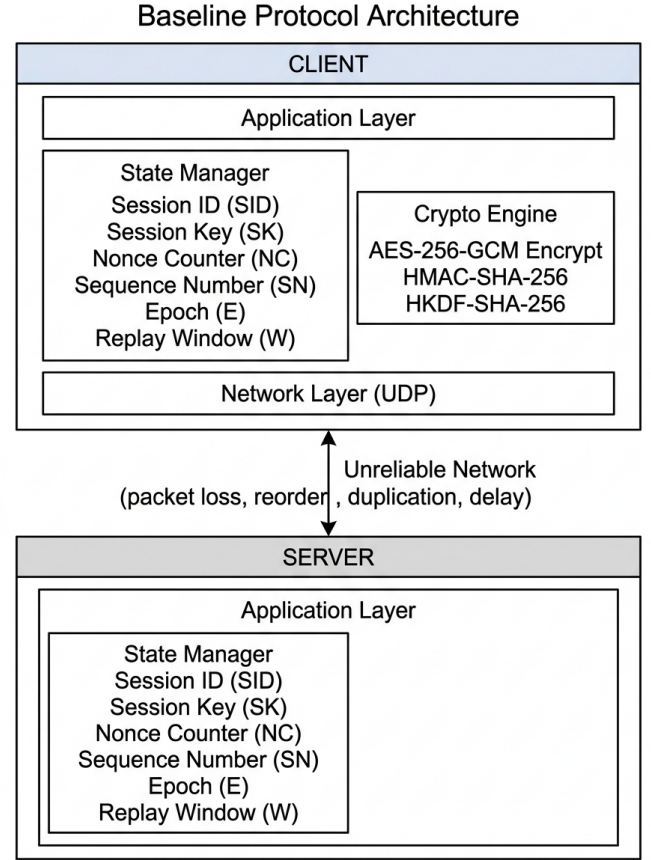


Fig. 1. Baseline architecture

custom protocol rather than modifying production protocols (TLS, SSH) to enable precise instrumentation and controlled experimentation. The following standard cryptographic primitives are used.

Authenticated Encryption: AES-256-GCM is used in the baseline to provide both confidentiality and integrity. Authenticated encryption combines these security properties in a single primitive [26]. [9].

Key Derivation: HKDF-SHA-256 is used for generating cryptographic keys. It follows the specification defined in RFC 5869 [24] and is also used within the TLS 1.3 key schedule [2].

Message Authentication: HMAC-SHA-256 is used for state authentication [25]. It is a standard message authentication code (MAC) construction which provides strong integrity protection and resistance against cryptographic collisions.

Figure 1 shows us the baseline architecture over an Unreliable UDP network. Both client and server maintain same protocol state by a state manager. The state manager mainly store the variables like session id, session key, nonce counter, sequence number, epoch and replay window. These state variables are required to ensure confidentiality, integrity, replay protection, and proper message ordering.

On the client side, a Crypto Engine performs cryptographic operations using AES-256-GCM for authenticated encryption, HMAC-SHA-256 for message authentication, and HKDF-SHA-256 for key derivation. Application data is passed through the application layer, secured by the cryptographic engine, and transmitted through the UDP-based network layer.

- For the baseline architecture some attack are preselected:
1. Intercept all network traffic.
 2. Drop packets in trasmission.
 3. Delay Packets.
 4. Reorder packets.
 5. Duplicate packets.
 6. Observe packets sizes, timing, and metadata.

The security goals of the system is:

Confidentiality: No plaintext information revealed to an eaves-dropper.

Integrity: No modification or alteration to ciphertext.

Authenticity: Only the legitimate user who have the session key can produce a valid message.

Replay resistance: The message which is detected a replay is rejected.

Secure Recovery: State recovery is only possible if both endpoints are detected legit.

A. System Architecture

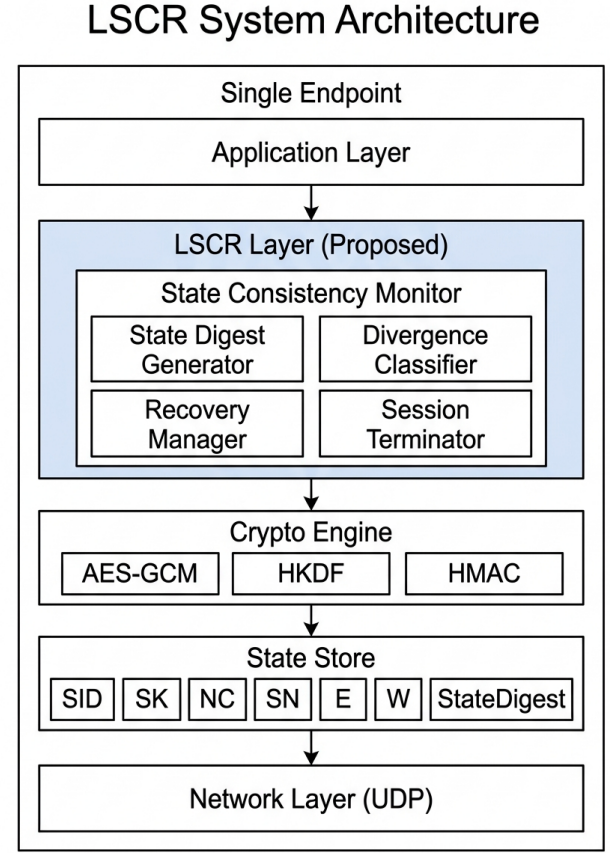


Fig. 2. Proposed architecture

Figure 2 show us the proposed architecture where the lscr layer is proposed mainly which introduce a separate monitoring layer to detect and mitigate cryptographic state desynchronization within a secure communication endpoint devices. The architecture organized into multiple layer the first layer is the application layer which collect the application data. Then the main LSCR layer, this layer contain four composed modules (State digest generator, divergence classifier, Recovery Manager, session Terminator). After the LSCR layer the crypto engine perform the cryptographic operation. Then finally the data transmitted through UDP based network layer.

Each endpoint maintains an extended state tuple:

$$S = (SID, SK, NC, SN_{send}, SN_{recv}, E, W, SD, T_{last}, F_{count})$$

Where :

SID : SessionIdentifier(128bitrandomvalueduringkeyexchange)

SK : Currentsessionkey(256 – bitAESkeybyHKDF)

NC : Noncecounter(64 – bitmonotonicallyincreasing)

SNsend : Outgoingsequencenumber(32 – bit)

SNrecv : Highestreceivedsequencenumber(32 – bit)

E : Epochcounter(16 – bit)

W : Anti – replaywindow

SD : StateDigest(256 – bitHMAC)

V. PROPOSED ARCHITECTURE

This proposed architecture mainly designed to satisfy the following requirements: 1. Protocol indepedency. 2. Lightweight Overhead 3.Authenticated Recovery. 4. Divergence Classification.

T_{last} : Timestamp of last successful message exchange
 F_{count} : Consecutive failure counter

B. Divergence Classification

Algorithm 1 Divergence Classification

```

1: function CLASSIFYDIVERGENCE( $S_{local}, msg_{remote}$ )
2:   if  $msg_{remote}.SID \neq S_{local}.SID$  then
3:     return IRRECOVERABLE
4:   end if
5:    $epoch\_diff \leftarrow msg_{remote}.E - S_{local}.E$ 
6:   if  $|epoch\_diff| > MAX\_EPOCH\_DRIFT$  then
7:     return IRRECOVERABLE
8:   end if
9:    $sn\_diff \leftarrow msg_{remote}.SN - S_{local}.SN_{recv}$ 
10:  if  $sn\_diff < -WINDOW\_SIZE$  then
11:    return STALE
12:  else if  $sn\_diff > MAX\_SN\_DRIFT$  then
13:    return SUSPICIOUS
14:  else if  $0 \leq sn\_diff \leq WINDOW\_SIZE$  then
15:    return RECOVERABLE_MINOR
16:  else if  $WINDOW\_SIZE < sn\_diff \leq MAX\_SN\_DRIFT$  then
17:    return RECOVERABLE_MAJOR
18:  end if
19:  if  $epoch\_diff = 0 \wedge |nc\_diff| > MAX\_NC\_DRIFT$  then
20:    return SUSPICIOUS
21:  end if
22:  return UNKNOWN
23: end function

```

C. Recovery Mechanism

Algorithm 2 Major Recovery

```

1: A detects RECOVERABLE_MAJOR divergence
2: A generates  $Nonce_{challenge}$ 
3: A sends  $STATE\_SYNC\_REQUEST$ 
4: B decrypts and verifies the request
5:  $\Delta_{SN} \leftarrow SN_{send}^A - SN_{recv}^B$ 
6:  $\Delta_{NC} \leftarrow NC_A - NC_B$ 
7: if  $0 \leq \Delta_{SN} \leq MAX\_RECOVERY\_GAP$  and  $0 \leq \Delta_{NC} \leq MAX\_NC\_RECOVERY\_GAP$  and  $E_A = E_B$  then
8:    $recovery\_decision \leftarrow SAFE$ 
9:    $Nonce_{response} \leftarrow HMAC-SHA256(SK, Nonce_{challenge} \parallel "sync\_ack")$ 
10:  B sends  $STATE\_SYNC\_RESPONSE(ACCEPT)$ 
11: else
12:    $recovery\_decision \leftarrow UNSAFE$ 
13:  B sends  $STATE\_SYNC\_RESPONSE(REJECT)$ 
14: end if
15: A verifies  $Nonce_{response}$ 
16: if  $response = ACCEPT$  then
17:   Update  $SN_{recv}^A$ 
18:   Advance replay window
19:   Verify nonce consistency
20:   Recompute state digest
21:   Send  $STATE\_SYNC\_CONFIRM$ 
22: else
23:   Terminate session
24:   Initiate fresh key exchange
25: end if
26: B receives  $STATE\_SYNC\_CONFIRM$ 
27: Both endpoints resume normal communication

```

TABLE II
DIVERGENCE CATEGORIES

Category	Cause	Action
RECOVERABLE MINOR	Packet loss/reordering in replay window	Advance replay window and continue communication
RECOVERABLE MAJOR	Significant packet loss	Perform authenticated state synchronization
STALE	Severely delayed packet	Discard packet and record event in logs
SUSPICIOUS	Abnormal state gap or attack attempt	Verify state consistency and expand if repeated
IRRECOVERABLE	Key mismatch, epoch mismatch	Terminate session and re-establish a new session
UNKNOWN	Unclassifiable divergence condition	Terminate session and re-establish communication

Algorithm 3 Message Send

```

1: function SENDMESSAGE( $plaintext, aad$ )
2:    $S.SN_{send} \leftarrow S.SN_{send} + 1$ 
3:    $S.NC \leftarrow S.NC + 1$ 
4:    $nonce \leftarrow ConstructNonce(S.E, SenderID, S.NC)$ 
5:    $full\_aad \leftarrow S.SID \parallel S.E \parallel S.SN_{send} \parallel aad$ 
6:    $(ciphertext, tag) \leftarrow AES-256-GCM.Encrypt(S.SK, nonce, plaintext, full\_aad)$ 
7:   if  $S.SN_{send} \bmod P = 0$  then
8:      $S.SD \leftarrow ComputeStateDigest(S)$ 
9:     AppendStateDigest( $ciphertext, S.SD$ )
10:  end if
11:   $packet \leftarrow full\_aad \parallel nonce \parallel ciphertext \parallel tag$ 
12:  NetworkSend( $packet$ )
13: end function

```

Algorithm 4 Message Receive

```

1: function RECEIVEMESSAGE(packet)
2:   Parse packet  $\rightarrow$  SID, E, SN, nonce, ciphertext, tag
3:   if SID  $\neq$  S.SID then
4:     DiscardAndLog("session mismatch")
5:     return FAILURE
6:   end if
7:   if E  $\neq$  S.E then
8:     classification  $\leftarrow$  ClassifyDivergence(S, packet)
9:     HandleDivergence(classification)
10:    return FAILURE
11:   end if
12:   full_aad  $\leftarrow$  SID||E||SN
13:   plaintext  $\leftarrow$ 
     AES-256-GCM.Decrypt(S.SK, nonce, ciphertext, full_aad, tag)
14:   if plaintext = NULL then
15:     S.Fcount  $\leftarrow$  S.Fcount + 1
16:     if S.Fcount >  $\theta_F$  then
17:       TriggerStateCheck()
18:     end if
19:     return FAILURE
20:   end if
21:   if not ReplayWindowCheck(S.W, SN) then
22:     DiscardAndLog("replay or out-of-window")
23:     return FAILURE
24:   end if
25:   UpdateReplayWindow(S.W, SN)
26:   S.SNrecv  $\leftarrow$  max(S.SNrecv, SN)
27:   S.Fcount  $\leftarrow$  0
28:   S.Tlast  $\leftarrow$  CurrentTime()
29:   if HasStateDigest(plaintext) then
30:     SDremote  $\leftarrow$  ExtractStateDigest(plaintext)
31:     SDlocal  $\leftarrow$  ComputeStateDigest(S)
32:     if SDremote  $\neq$  SDlocal then
33:       classification  $\leftarrow$  ClassifyDivergence(S, packet)
34:       HandleDivergence(classification)
35:     end if
36:   end if
37:   return plaintext
38: end function

```

D. Experimental Scenarios

TABLE III
EXPERIMENTAL NETWORK SCENARIOS

Scenario	Network Condition	tc/netem Configuration
S1	Normal (baseline)	No impairment
S2	1% packet loss	netem loss 1%
S3	5% packet loss	netem loss 5%
S4	10% packet loss	netem loss 10%
S5	Packet duplication (5%)	netem duplicate 5%
S6	Packet reordering (25%, gap 5)	netem reorder 25% gap 5%
S7	Packet delay (50 ms $\pm$ 20 ms)	netem delay 50ms and 20ms
S8	Connection interruption	Programmatic link drop (5 s)
S9	Endpoint restart	Programmatic state reset
S10	Replay attack	External replay injection
S11	Combined: 5% loss + reordering	netem loss 5% reorder 25% gap 5
S12	Combined: 5% loss + interruption	netem loss 5% + programmatic drop

Each scenario is executed with 1000 message per trail and 10 trail per scenario for both baseline and proposed system and the message rate is 100 message/second.

VI. RESULT AND DISCUSSION

A. Result

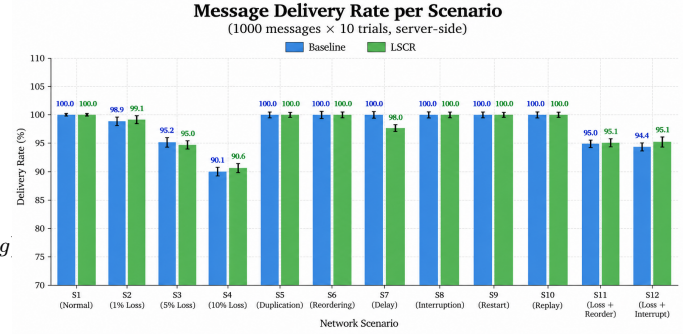


Fig. 3. Message Delivery and State Divergence

Observation:

S1: Zero message loss for both architecture
S2-S4: Message delivery rates closely match the expected loss rate (0.9%, 5%, 9.4%).
S5: All 1000 message were delivered successfully. An average of 50.8 for baseline and 4.6 for LSCR duplicate packet were received and rejected without any authentication failures.
S6: 64 entry anti replay window successfully reordered packets in both system with 100% delivery rate.
S7: The baseline architecture achieved 100% delivery, while LSCR faced 2% message loss due to the session failures by state check timeout interactions. All 10 LSCR trials recorded session failures (100 total across server-side summaries).
S8-S9: Both system achieved 100% delivery rate after session re establishment in S8. Both architecture triggered 1 session failure per trail.
S10: All legit message were delivered. No architecture accepted replay.
S11: The result of S11 scenario match's closely with S3 indicating that reordering in the window does not relate with loss.
S12: The proposed architecture show 0.7% better result then baseline architecture which mean LSCR session re establishment after interruption is marginally more efficient.

Session Continuity and Recovery by Scenario					
Scenario	Divergence Classification	Baseline Session Failures	LSCR Session Failures	LSCR Recovery Attempts	Session Re-establishments
S1 (Normal)	None	0	0	0	0
S2 (1% loss)	None (within window)	0	0	0	0
S3 (5% loss)	None (within window)	0	0	0	0
S4 (10% loss)	None (within window)	0	0	0	0
S5 (Duplication)	None (duplicates rejected)	0	0	0	0
S6 (Reordering)	None (within window)	0	0	0	0
S7 (Delay)	Timeout-induced	0	100 (10/trial)	0	10
S8 (Interruption)	Programmatic reset	0	0	0	0
S9 (Restart)	IRRECOVERABLE (state reset)	10 (1/trial)	10 (1/trial)	0	10
S10 (Replay)	None	0	0	0	0
S11 (Loss + Reorder)	None/Minor	0	1	0	0
S12 (Loss + Interrupt)	Programmatic reset	0	0	0	0

Fig. 4. Session Continuity and Recovery

Metric	Baseline	LSCR (Normal)
CPU utilization — client (%)	3.9	5.1
CPU utilization — server (%)	2.1	2.7
Memory utilization — client (KB)	29,088	29,120
Memory utilization — server (KB)	28,976	29,313
Crypto ops per message (normal)	1 AES-GCM	1 AES-GCM + 0.02 HMAC
Crypto ops per message (state check)	N/A	1 AES-GCM + 1 HMAC
Avg. encrypt latency per message (μ s)	239.9 $\pm$ 102.5	267.5 $\pm$ 196.5
Encrypt latency overhead (%)	—	+11.5%

Fig. 5. Computational Overhead

Figure 5 shows us the computational overhead of LSCR during normal operation. The client side CPU utilization increased by 1.2% (from 3.9% to 5.1%), and memory utilization increased by 32 KB (+0.1%). The average per message encryption latency increased by 27.6 μ s (+11.5%), due to the additional HMAC-SHA-256 computation for the state digest every 50 messages (TATE_CHECK_PERIOD = 50) and the type byte payload covering on every message. The server side overhead is similarly small, with CPU increasing by 0.6 percentage points and memory by 337 KB.

TABLE IV
BASELINE VS PROPOSED ARCHITECTURE

Dimension	Scenario	Baseline	LSCR	Delta
Message delivery (%)	S1 (Normal)	100.0%	100.0%	0.0%
Message delivery (%)	S2 (1% loss)	98.9 $\pm$ 0.2	99.1 $\pm$ 0.2	+0.2%
Message delivery (%)	S3 (5% loss)	95.2 $\pm$ 0.3	95.0 $\pm$ 0.6	-0.1%
Message delivery (%)	S4 (10% loss)	90.1 $\pm$ 0.6	90.6 $\pm$ 0.5	+0.5%
Message delivery (%)	S5 (Duplication)	100.0%	100.0%	0.0%
Message delivery (%)	S6 (Reordering)	100.0%	100.0%	0.0%
Message delivery (%)	S7 (Delay)	100.0%	98.0 $\pm$ 0.4	-2.0%
Message delivery (%)	S8 (Interruption)	100.0%	100.0%	0.0%
Message delivery (%)	S9 (Restart)	100.0%	100.0%	0.0%
Message delivery (%)	S10 (Replay)	100.0%	100.0%	0.0%
Message delivery (%)	S11 (Loss + Reorder)	95.0 $\pm$ 0.4	95.1 $\pm$ 0.7	+0.1%
Message delivery (%)	S12 (Loss + Interrupt)	94.4 $\pm$ 0.4	95.1 $\pm$ 0.6	+0.7%
Security violations	All scenarios	0	0	—
Encrypt latency (μ s)	S1 (Normal)	239.9	267.5	+11.5%
Communication overhead	S1 (Normal)	72.0 B/msg	69.6 B/msg	-3.3%
CPU overhead — client	S1 (Normal)	3.9%	5.1%	+1.2 pp
Memory overhead — client	S1 (Normal)	29,088 KB	29,120 KB	+0.1%

B. Discussion

1) *Security Discussion:* The experimental analysis of the proposed architecture confirms that it preserve all security properties of the baseline under the Dolev-Yao adversarial model assumptions [22]. for all the scenarios. Zero nonce reuse events[27], zero replay acceptances, and zero invalid message acceptances were reordered across 480 experiment. This experiment result validate the core security claim that the LSCR layers does not weaken the AEAD security. The nonce counter along with the increasing counter and sender ID spaces prevents nonce reuse. Under the S10 scenario all replayed packets were correctly rejected by the anti replay window.

2) *Availability Discussion:* Figure 3 shows that the proposed and baseline architecture perform similarly under most

of the scenarios but the proposed architecture show a small improvement under the S4 with 10% loss: +.5%, S12 with loss+interruption: +.7% and S2 with .2%. However, proposed architecture showed a 2% delivery decrease under the S7 scenario. All the 10 LSCR trails under S7 record multiple server side session failure which indicate that the time out based state checking is more sensitive to sustained delay. This finding highlight the importance of the parameter sensitivity and suggest that the state check timeout parameter should be tuned relative to the expected network delay. Another key observation is that no cryptographic state desynchronization was triggered in the testes scenarios. The packet loss scenario occurs because of the messages's non delivery but not for the state desynchronization. This means the LSCR recovery protocol was not implemented during the experiment. The experiment architecture successfully validate the LSCR mechanism in the common network condition but the recovery protocols validation under more extreme stress condition remain untouched.

3) *Performance Discussion:* The performance overhead of the proposed architecture is minimal and consistent with our design goal which is lightweight operation. The increase of per message encryption latency occurs because of the HMAC-SHA-256 state digest parameter computation in every 50 messages and also the 1-byte type prefix added to every payload. The latency of 27.6 μ s per message is negligible for most of the real life application. The memory overhead of 32 KB on the client side indicates the additional state maintained by the LSCR layer. The CPU overhead of 1.2% is also in the acceptable bounds for a security enhancing layer.

The communication overhead is slightly negative (-3.3%), this is because of the proposed architecture message construction. The baseline message encrypts the full plaintext directly which produced a 72 byte packets but the proposed architecture prepends a 1 byte type prefix to the payload first resulting in 69 byte normal packets and 101 byte state check packets.

TABLE V
COMPARISON OF LSCR WITH EXISTING STATE MANAGEMENT MECHANISMS

Feature	IPsec	DTLS	WG	Signal	RFID	Propose Framework
Protocol independence	No	No	No	No	No	Yes
State divergence detection	Implicit	Implicit	Implicit	MAC fail	Server check	Explicit digest
Divergence classification	None	None	None	None	None	Multi-level
Authentication	SA	TLS key	Session key	Chain key	Shared secret	HMAC session key
Replay protection	Window	Window	Window	Ratchet	Counter	Window + digest
Partial recovery	Discard	HS retransmit	Re-handshake	Ratchet	Old key fallback	Auth. state sync
Fail-safe behavior	Discard	Reconnect	Discard	N/A	N/A	Terminate + rekey
Network fault tolerance	Limited	High	Limited	Low	N/A	High
Computational overhead	Low	Low-Med	Low	Medium	Very low	Low/Med
Communication overhead	None	Epoch ovhd.	None	DH ratchet	Updates	State digest
Formal security analysis	RFC	Formal	CryptoVerif	Formal	BAN	Informal
Experimental evaluation	Deploy.	Limited	Deploy.	N/A	Sim	Testbed

VII. LIMITATION AND CONCLUSION

A. Limitation and Future Work

1) *Limitation*: The experiment is conducted by using a controlled single machine testbed with tc/netem network simulation. Real world network behavior may differ from the simulated conditions. Also the security analysis mainly relies on the standard security protocol although these mechanisms are well established but not fully unconditional. Beside these this paper has no hardware implementation that address timing side channels, power analysis or other physical side channel attack. Moreover the evaluation primarily consider point to point session which means it is to scalable with many concurrent session.

2) *Future Work*:

- Integration with TLS 1.3
- Integration with DTLS 1.3
- Adaptation for MQTT and CoAP
- Evaluation on IoT and Edge Devices
-
- Formal Verification using Tamarin/ProVerif similar to formal analyses performed for TLS 1.3 [23]
- Large-Scale Internet Deployment Testing
- Evaluation in Heterogeneous Networks
- Secure Persistent State Storage
- Multi-Party Communication Extension
- Adaptive Parameter Tuning
- Comparison with QUIC

B. Conclusion

This paper mainly investigates the problem of cryptographic state desynchronization in a secure network communication caused by the network event like packet loss, duplication, reordering, delay, and many more. By a in depth literature review of transport security protocols, authentication protocols, desynchronization attacks recovery mechanisms, the research gap identified. Existing approaches are protocol specific, attack specific or domain specific but lack of a systematic, protocol independent mechanism for detection, classification and recovery from network induced cryptographic state divergence

Then we proposed a lightweight state consistency and recovery architecture which provide authentication state digest, multi level divergence classification and also a recoverable protocol form irrecoverable states. We implemented and evaluated the mechanism in a controlled experiment testbed using a single kali linux tc/netem for network simulation across 12 scenarios with 1000 message per trails, 10 trails per scenario. From 480 controlled experiment summaries we get: zero nonce reuse events, zero replay acceptances, zero invalid message acceptance also the performance overhead were minimal and all performance overhead were in range. Under the packet loss scenario the proposed architecture achieves improved message delivery rates relative to the baseline architecture

VIII. REFERENCES

- [1] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, Nov. 1976. DOI: 10.1109/TIT.1976.1055638.
- [2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [3] S. Kent, "IP Encapsulating Security Payload (ESP)," RFC 4303, Internet Engineering Task Force, Dec. 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4303>
- [4] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," RFC 4301, Internet Engineering Task Force, Dec. 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4301>
- [5] E. Rescorla, H. Tschofenig, and N. Modadugu, "The Datagram Transport Layer Security (DTLS) Protocol Version 1.3," RFC 9147, Internet Engineering Task Force, Apr. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9147>
- [6] J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel," in *Proc. Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2017. DOI:10.14722/ndss.2017.23160.
- [7] P. Gope and T. Hwang, "A Realistic Lightweight Anonymous Authentication Protocol for Securing Real-Time Application Data Access in Wireless Sensor Networks," *IEEE Transactions on Industrial Electronics*, vol. 63, no. 11, pp. 7124-7132, Nov. 2016. DOI:10.1109/TIE.2016.2585081.
- [8] M. Safkhani and N. Bagheri, "Generalized Desynchronization Attack on UMAP: Application to RCIA, KMAP, SLAP, and SASI+ Protocols," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E96-A, no. 7, pp. 1611-1619, 2013. DOI: 10.1587/transfun.E96.A.1611.
- [9] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, National Institute of Standards and Technology, Nov. 2007. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-38d/final>
- [10] S. Gueron, A. Langley, and Y. Lindell, "AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption," RFC 8452, Internet Research Task Force, Apr. 2019. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8452>
- [11] T. van Deursen and S. Radomirovic, "Attacks on RFID Protocols," *Cryptology ePrint Archive*, Report 2008/310, 2009. [Online]. Available: <https://eprint.iacr.org/2008/310>
- [1] W. Diffie and M. Hellman, "New directions in cryptography,"

- [12] Y. Sheffer, P. Saint-Andre, and T. Fossati, "Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)," RFC 9325, Internet Engineering Task Force, Nov. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9325>
- [13] T. Perrin, "The Noise Protocol Framework," Revision 34, Jul. 2018. [Online]. Available: <https://noiseprotocol.org/noise.html>
- [14] T. Perrin and M. Marlinspike, "The Double Ratchet Algorithm," Signal Foundation, Nov. 2016. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>
- [15] G. Avoine, M. A. Bingol, X. Carpent, and S. Yalcin, "Privacy-Friendly Authentication in RFID Systems: On Sublinear Protocols Based on Symmetric-Key Cryptography," *IEEE Transactions on Mobile Computing*, vol. 12, no. 10, pp. 2037-2049, Oct. 2013. DOI:10.1109/TMC.2012.171.
- [16] A. Idriss, H. Bayoussef, and A. Asimi, "A PUF-Based Lightweight Mutual Authentication Protocol for IoT Devices Using RFID Tags," *Electronics*, vol. 13, no. 12, Art. no. 2321, Jun. 2024. DOI: 10.3390/electronics13122321.
- [17] M. Brinkmann, C. Dresen, R. Merget, D. Poddebniak, J. Mueller, J. Somorovsky, J. Schwenk, and S. Schinzel, "The Opossum Attack: Exploiting Cross-Protocol Vulnerabilities in TLS," presented at the 34th USENIX Security Symposium, Aug. 2025. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25>
- [18] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Taquist, "Automata-Based Automated Detection of State Machine Bugs in Protocol Implementations," in *Proc. 30th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, Feb. 2023. DOI:10.14722/ndss.2023.24274.
- [19] A. Firdous, S. Shahzad, and M. Kamran, "Dynamic Session Enhanced Key Protocol for IoT Security," *arXiv preprint*, arXiv:2404.xxxxx, 2024.
- [20] J. Granjal, E. Monteiro, and J. Sa Silva, "Security for the Internet of Things: A Survey of Existing Protocols and Open Research Issues," *IEEE Communications Surveys and Tutorials*, vol. 17, no. 3, pp. 1294-1312, Third Quarter 2015. DOI: 10.1109/COMST.2015.2388550.
- [21] K. Angrisani, A. Ferrara, and M. De Benedictis, "L-TESLA-CoAP: Lightweight Authentication for Constrained IoT Devices," *Sensors*, vol. 23, no. 14, Art. no. 6438, Jul. 2023.
- [22] D. Dolev and A. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198-208, Mar. 1983. DOI: 10.1109/TIT.1983.1056650.
- [23] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A Comprehensive Symbolic Analysis of TLS 1.3," in *Proc. 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Dallas, TX, USA, 2017, pp. 1773-1788. DOI:10.1145/3133956.3134063.
- [24] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5869>
- [25] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2104>
- [26] M. Bellare and C. Namprempre, "Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm," *Journal of Cryptology*, vol. 21, no. 4, pp. 469-491, Oct. 2008. DOI: 10.1007/s00145-008-9026-x.
- [27] N. Ferguson, "Authentication Weaknesses in GCM," Comments on GCM submitted to NIST, May 2005. [Online]. Available: <https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/bcm/comments/cwc-gcm/ferguson2.pdf>